

## Aula 3 — Setup + Unitários + Integração

*"Testa a lógica isolada. Depois testa as peças juntas."*

Jest · React Native Testing Library · renderHook

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 22/06/2026

## Recap da Aula 2

- **Fork + PR no GitHub** — como entregar atividades (fluxo completo, CI embutida nos PRs)
- **Pirâmide de testes** — por que mobile tem mais testes manuais (Khorikov)
- **Testes manuais estruturados**: FEW HICCUPPS (Bolton) + tours (Whittaker) + Charter/SBTM (Bach)
- **Exercício ao vivo**: Organic Maps — 3 heurísticas + Anti-Social Tour
- **Template de bug report mobile** — SO, device, build number, frequência de reprodução

**Setup RN + Jest/RNTL ficou para hoje** — começa agora.

**Atividade 1** — prazo era 10/06. Entregue?

## Agenda da aula (3h30)

1. **Bloco 1 (75min)** — Setup + testes unitários: funções puras, lógica de estado, cobertura
2. **Intervalo (30min)**
3. **Bloco 2 (75min)** — Testes de integração: RNTL com contexto, navegação, API mock, `renderHook`
4. **Esquentar (15min)** — Atividade 2 + prévia da Aula 4 (Maestro E2E)

## Objetivos de aprendizagem

- **Configurar** o projeto RN e rodar a suíte de testes pela primeira vez
- **Implementar** testes de domain puro com Jest (funções puras e lógica de estado)
- **Implementar** testes de componentes com React Native Testing Library
- **Configurar** cobertura mínima via Istanbul + ler o relatório
- **Implementar** testes de integração: componente com navegação + API mockada
- **Usar** `renderHook` para testar hooks com dependências

## Teste manual vs automatizado — em mobile

Na Aula 2 você fez **teste manual/exploratório**. Hoje começa o **automatizado**. Não competem — se completam.

|            | Manual / Exploratório (Aula 2)            | Automatizado (de hoje em diante)            |
|------------|-------------------------------------------|---------------------------------------------|
| Forte em   | usabilidade, UX, tela nova, bug visual    | regressão, regra de negócio, fluxo repetido |
| Frequência | uma vez · exploração humana               | toda vez · a cada PR no CI                  |
| Em mobile  | gestos, sensores, permissões, device real | cobrir a fragmentação (muitos OS/devices)   |
| Custo      | tempo humano a cada rodada                | escreve 1x, roda pra sempre                 |

Automatize o **repetitivo e estável** (regressão); explore **manualmente** o novo e o subjetivo. A fragmentação mobile torna a automação essencial — mas gesto/sensor ainda pede mão humana.

## O que a pesquisa recomenda (mobile)

**Cruz, Abreu & Lo (2019)** — *"Guess what, test your app!"* (Empirical Software Engineering)

Analysaram centenas de apps Android open-source: a **maioria tem pouco ou nenhum teste automatizado**. Os que testam evoluem com **menos bugs e regressões**.

**Recomendação prática — converge com Khorikov e a pirâmide:**

- Comece pela **lógica de negócio** (unit) — barato, rápido, alto retorno
- **Automatize a regressão** e rode no **CI** a cada mudança
- Reserve **E2E** (caro e frágil) só pros **fluxos críticos**

Linaires-Vásquez et al. (2017, ICSME) reforça: teste mobile deve ser **contínuo e integrado ao desenvolvimento** — não um esforço pontual no fim.

## Glossário rápido — você vai ouvir isso hoje

- **Flaky** — teste que ora passa, ora falha **sem mudar o código**. Falso alarme (causas: tempo, animação, ordem, rede). Mina a confiança na suíte.
- **Renderizar** — "desenhar" o componente **em memória** pra inspecioná-lo no teste, sem abrir o app no simulador.
- **Cobertura (coverage)** — % do código que os testes executam. Cobertura alta  $\neq$  bons testes.
- **Asserção ( `expect` )** — a verificação que faz o teste passar ou falhar: `expect(x).toBe(y)`.
- **Regressão** — algo que funcionava e quebrou; testes de regressão pegam isso cedo.

**Mock / stub / spy** têm slide próprio mais à frente (*Test Doubles*).

## Por que unit antes de integração?

**E2E · Maestro** ← Aula 4

**Integração · RNTL** ← Bloco 2 de hoje

**Unitários · Jest** ← Bloco 1 de hoje

! Não dá pra depurar integração sem entender o unit. E não faz sentido E2E sem integração estável por baixo.



## JS/TS puro vs React — dois mundos, dois custos

|                 | JS / TS puro                                                                   | React (RN)                                                        |
|-----------------|--------------------------------------------------------------------------------|-------------------------------------------------------------------|
| O que é         | lógica, dados, regras                                                          | componente que desenha UI                                         |
| Exemplo         | <code>isFavorito</code> , <code>posterUrl</code> , <code>favoritesStore</code> | <code>&lt;MovieCard/&gt;</code> , <code>&lt;MovieList/&gt;</code> |
| Tem tela?       | não                                                                            | sim ( <code>&lt;View&gt;</code> , <code>&lt;Text&gt;</code> )     |
| Pra testar      | Jest — só Node                                                                 | RNTL — renderiza em memória                                       |
| Tempo por teste | ~1 ms                                                                          | ~dezenas de ms                                                    |
| Flaky?          | praticamente 0                                                                 | pode flakar (render/async)                                        |

Testamos nesta ordem: **pura** → **estado** → **componente** — é a sequência dos próximos slides. E2E só na Aula 4 (segundos por teste). Regra de negócio mora no **puro**; UI é a casca.

## Unit testing RN — pirâmide



### ● Domain puro

função/regra (TS) → Jest

### ● Hooks / estado

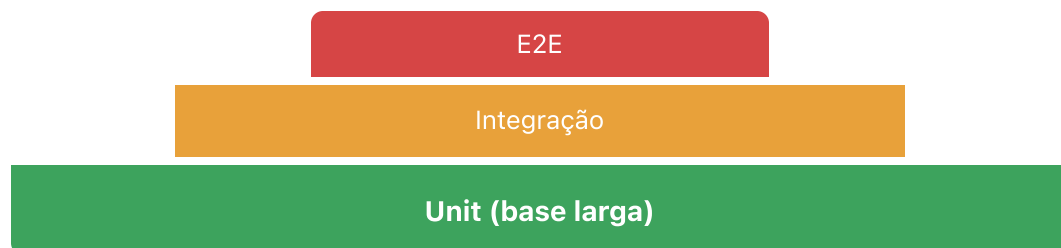
lógica de estado, efeitos → renderHook

### ● Componentes

UI / comportamento → RNTL (precisa render)

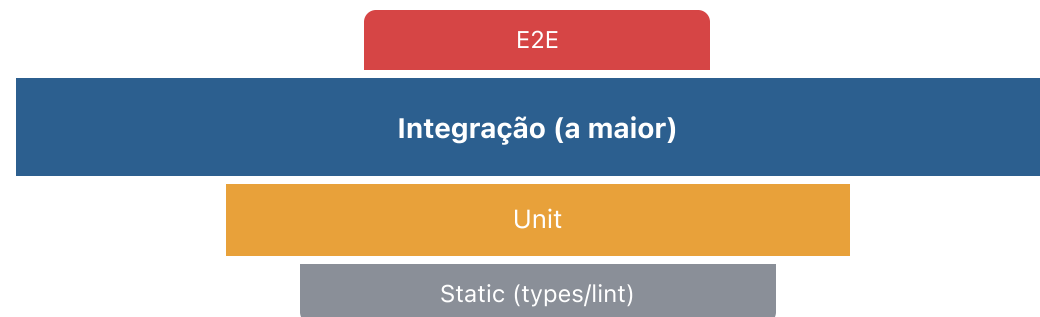
## Pirâmide vs Troféu — conflitam?

### Pirâmide



Cohn (2009) · Fowler  
maximize **unit** isolado

### Troféu



Kent C. Dodds (2018)  
maximize **integração**

Não é guerra — é ênfase. Os dois cortam E2E (caro/frágil). A diferença é o meio: em **UI/RN** o maior valor está na **integração** (RNTL com deps reais). Dodds: *"quanto mais o teste se parece com o uso real, mais confiança ele dá"*. Por isso hoje fazemos **unit (Bloco 1) + integração (Bloco 2)** — E2E fica pra Aula 4.

## Lógica pura — da história ao TypeScript

**História:** o ❤️ do filme fica **cheio** quando é favorito e **vazio** quando não é. Pra saber qual desenhar, o app faz uma pergunta simples: esse *filme está na lista de favoritos*?

**Algoritmo (pense antes de abrir o editor):**

- recebe a **lista de ids favoritados** e o **id do filme**
- responde **sim** se o id está na lista, **não** se não está

```
// regra pura – entra (lista, id), sai (sim/não). Sem React, sem tela.  
function isFavorito(ids: number[], id: number): boolean {  
  return ids.includes(id);  
}
```

**Como testar — só pensar em entrada → saída:**

- `isFavorito([1, 2, 3], 2) → true`
- `isFavorito([1, 2, 3], 9) → false`

Sem simulador, sem mock. Jest roda isso em < 1ms. É a base da `favoritesStore` (próximo slide) que você testa na Atividade 2.

## Domain layer — Jest puro

Lógica de negócio isolada de React e do nativo. Teste mais barato e mais estável.

```
// src/utils/posterUrl.ts
export function posterUrl(path: string, size: 'w185' | 'w500' = 'w500') {
  return `https://image.tmdb.org/t/p/${size}${path}`;
}
```

```
// src/utils/__tests__/posterUrl.test.ts
import { posterUrl } from '../posterUrl';

describe('posterUrl', () => {
  it('gera URL com tamanho padrão w500', () => {
    expect(posterUrl('/abc.jpg')).toBe('https://image.tmdb.org/t/p/w500/abc.jpg');
  });

  it('respeita o tamanho passado', () => {
    expect(posterUrl('/abc.jpg', 'w185')).toBe('https://image.tmdb.org/t/p/w185/abc.jpg');
  });
});
```

## Domain layer — lógica de estado

A `favoritesStore` **já vem pronta** — trate como caixa-preta. Seu trabalho é **testar o comportamento**, não reimplementar:

```
// src/store/favoritesStore.ts — o contrato (o que cada método faz)
add(id)           // adiciona o id
remove(id)        // remove o id
toggle(id)        // alterna: adiciona se ausente, remove se presente (a regra isFavorito)
isFavorite(id)    // → true / false
clear()           // esvazia a lista
```

```
// __tests__/favoritesStore.test.ts ← ISSO é o seu trabalho
import { useFavoritesStore } from '../favoritesStore';
const store = () => useFavoritesStore.getState();

beforeEach(() => store().clear()); // store é singleton: limpe entre testes

test('toggle alterna — adiciona e depois remove', () => {
  store().toggle(42); // ausente → adiciona
  expect(store().isFavorite(42)).toBe(true);
  store().toggle(42); // presente → remove
  expect(store().isFavorite(42)).toBe(false);
});
```

# Test Doubles — os que você usa hoje (Meszaros, 2007)

**Test double** = objeto falso que substitui uma dependência real (rede, navegação, módulo nativo) pro teste não depender dela.

| Tipo | O que faz                                           | No app de filmes (Atividade 2)                                                                        |
|------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Stub | Devolve resposta fixa                               | <code>jest.mock('@services/api')</code> → API devolve filmes falsos<br>( <code>popularMovies</code> ) |
| Spy  | Grava as chamadas pra inspecionar                   | <code>mock do useNavigation</code> no <code>MovieCard</code>                                          |
| Mock | Spy com expectativa: falha se não for chamado certo | <code>expect(navigate).toHaveBeenCalledWith('Detail', { id: 42 })</code>                              |

*Também existem **Dummy** (só preenche assinatura) e **Fake** (implementação simplificada) — menos comuns aqui. Nome vem do cinema: *stunt double* (dublê). Na prática hoje: `jest.mock` (Stub) + `jest.fn / spyOn` (Spy/Mock) — detalhado no próximo slide.*

## Mockando dependências — jest.mock

```
// src/services/movieService.ts
import { api } from './api';
export async function fetchPopularMovies() {
  const { data } = await api.get('/movie/popular');
  return data.results;
}

// __tests__/movieService.test.ts
import { fetchPopularMovies } from '../movieService';
import { api } from '../api';

jest.mock('../api');
const mockedGet = api.get as jest.Mock;

it('retorna a lista de results da API', async () => {
  const fakeMovies = [{ id: 1, title: 'Matrix' }];
  mockedGet.mockResolvedValue({ data: { results: fakeMovies } });

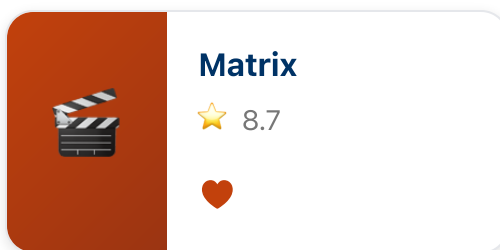
  const movies = await fetchPopularMovies();

  expect(movies).toEqual(fakeMovies);
  expect(mockedGet).toHaveBeenCalledWith('/movie/popular');
});
```



## Componente — da tela ao React

▢ O que o produto pediu



🔗 O componente React Native

```
<TouchableOpacity onPress={navigate}>
  <Image source={{ uri: posterUrl(...) }} />
  <View>
    <Text>{movie.title}</Text>
    <Text>★ {movie.vote_average}</Text>
  </View>
  <Text>♥</Text>
</TouchableOpacity>
```

**TouchableOpacity** → card clicável

**Image** → poster

**Text** → título e nota

♥ → botão favoritar

Mesma história de antes: o ♥ usa a lógica de **favoritos** ( `isFavorito` / `favoritesStore` ). React só **declara** o que aparece — o teste RNTL verifica sem simulador.

## Componentes — React Native Testing Library

Testa comportamento de tela — o que o usuário vê e toca.

```
import { render, screen, fireEvent } from '@testing-library/react-native';
import MovieCard from '../src/components/MovieCard';

const mockNavigate = jest.fn();
jest.mock('@react-navigation/native', () => ({
  useNavigation: () => ({ navigate: mockNavigate }),
}));
const movie = { id: 42, title: 'Matrix', poster_path: '/m.jpg', vote_average: 8.7 };

it('mostra título e nota', () => {
  render(<MovieCard movie={movie} />);
  expect(screen.getByText('Matrix')).toBeTruthy();
  expect(screen.getByText('★ 8.7')).toBeTruthy();
});

it('navega ao tocar no card', () => {
  render(<MovieCard movie={movie} />);
  fireEvent.press(screen.getByText('Matrix'));
  expect(mockNavigate).toHaveBeenCalledWith('Detail', { id: 42, title: 'Matrix' });
});
```

## RNTL — queries por prioridade

| Query                       | Prioridade | Por quê                               |
|-----------------------------|------------|---------------------------------------|
| <code>getByRole</code>      | ✓ Alta     | Reflete acessibilidade real           |
| <code>getByLabelText</code> | ✓ Alta     | Segue <code>accessibilityLabel</code> |
| <code>getByText</code>      | ● Média    | Frágil a mudanças de copy             |
| <code>getById</code>        | ● Baixa    | Escapa da UI real — última opção      |

```
// ✓ Prefira
screen.getByRole('button', { name: 'Adicionar favorito' })

// ● Evite como primeiro recurso
screen.getById('fav-button')
```

`testID` é escape hatch, não padrão. Priorize queries semânticas.

## Cobertura — Jest + Istanbul

```
npm test -- --coverage
open coverage/lcov-report/index.html
```

```
// jest.config.js - gate no CI
module.exports = {
  preset: 'react-native',
  collectCoverageFrom: ['src/**/*.{ts,tsx}', '!src/**/*.d.ts'],
  coverageThreshold: {
    global: { branches: 70, functions: 70, lines: 70, statements: 70 },
  },
};
```

| Métrica           | Mede                                |
|-------------------|-------------------------------------|
| Statements        | % de instruções executadas          |
| Branches          | % de caminhos ( if/else , && , ?: ) |
| Lines / Functions | % de linhas / funções chamadas      |

## Lendo o relatório

|                   |                   |        |        |        |           |
|-------------------|-------------------|--------|--------|--------|-----------|
| Tests:            | 6 passed, 6 total |        |        |        |           |
| -----             | -----             | -----  | -----  | -----  | -----     |
| File              | % Stmt            | % Brnc | % Func | % Line | Uncovered |
| -----             | -----             | -----  | -----  | -----  | -----     |
| favoritesStore.ts | 100               | 50     | 100    | 100    | 12        |

- Branch 50% = você testou o toggle que **adiciona**, mas não o que **remove** → a linha 12 (caminho do remove) ficou de fora
  - Tests passed ≠ cobertura alta — são dimensões diferentes
- Dá pra ter 100 testes verdes cobrindo 10% do código. Cobrir os **dois caminhos** do toggle leva o branch a 100%.

## O que faz um teste bom?

```
// ❌ sem assertion – cobre a linha, não prova nada
it('renderiza', () => { render(<MovieCard movie={movie} />); });

// ✅ verifica o que o usuário vê
it('exibe o título do filme', () => {
  render(<MovieCard movie={movie} />);
  expect(screen.getByText('Matrix')).toBeTruthy();
});
```

```
// ❌ só verifica se o mock foi chamado – ignora o resultado
expect(api.get).toHaveBeenCalledTimes();

// ✅ verifica o que a função retornou de fato
const movies = await fetchPopularMovies();
expect(movies).toEqual([ { id: 1, title: 'Matrix' } ]);
expect(api.get).toHaveBeenCalledWith('/movie/popular');
```

**Teste bom: uma razão pra falhar, uma coisa verificada, nome que documenta a intenção.**

## Hands-on Bloco 1 — o app que vamos testar

**Catálogo de filmes (RN + Expo).** Implementado — você só escreve os testes.

```
cd puc-iec-testes-aplicacoes-mobile/exercicios/02-setup-suite-unitaria/exercicio
npm install && npm test # posterUrl já passa verde
```

Frentes de teste (sem nem rodar o app):

- `posterUrl` — helper que monta URL da imagem ← **começa aqui**
- `isTokenError` — classifica erro de autenticação da API
- `favoritesStore` · `counterStore` — lógica de estado (Zustand por baixo)
- **MovieCard (RNTL)** — renderiza e responde ao toque

## Roteiro hands-on Bloco 1 (75min)

1. `npm test verde + ler posterUrl.test.ts` (modelo já pronto) — **5min**
2. `isTokenError` — função pura, escreve junto — **10min** ← confiança primeiro
3. `favoritesStore` — lógica de estado (add/remove/toggle) — **15min**
4. `MovieCard` (RNTL) — render do título/nota + `press` → navega — **20min** ★
5. `counterStore` — vocês sozinhos — **5min**
6. `npm run test:coverage` → ler relatório e discutir — **5min**
7. Mutation testing — Stryker demo rápida — **5min** (se sobrar tempo)

Bônus: `fetchPopularMovies` com `jest.mock` da api (+10min)



## Pitfalls unit testing

- ✗ **Sleep em unit test** — anti-pattern absoluto
- ✗ **Mock de tudo** — acopla ao "como", não ao "o quê"
- ✗ **Test names sem intenção** ( `testFunc1` ) — renomeie pra revelar o contrato
- ✗ **Coverage como métrica única** — alta cobertura + mutation score baixo = ilusão
- ✗ **Teste sem `expect`** — passa verde, não prova nada
- ✗ **Testar detalhes de implementação** — quebrará em refactor sem bug real

 **Intervalo — 30min**

## Bloco 2 — Testes de Integração

Unitário testa peças isoladas. Integração testa **peças trabalhando juntas**.

|              | Unitário                 | Integração                              |
|--------------|--------------------------|-----------------------------------------|
| O que isola? | tudo — moca dependências | só o que está fora do app (API, device) |
| Exemplo      | favoritesStore sozinho   | MovieList + store + navegação           |
| Confiança    | lógica interna           | fluxo do ponto de vista do usuário      |
| Velocidade   | muito rápido             | rápido (ainda sem simulador)            |

| RNTL é a ferramenta certa: renderiza a árvore real de componentes, mas sem simulador.

## Integração: componente + navegação

Para testar componentes que usam `useNavigation`, envolva com `NavigationContainer`:

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { render, screen, fireEvent } from '@testing-library/react-native';
import MovieList from '../src/screens/MovieList';
import MovieDetail from '../src/screens/MovieDetail';

const Stack = createNativeStackNavigator();

function AppNavigator() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="MovieList" component={MovieList} />
        <Stack.Screen name="Detail" component={MovieDetail} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

it('tap no card navega pra tela de detalhe', async () => {
  render(<AppNavigator />);
  fireEvent.press(await screen.findByText('Matrix'));
  expect(await screen.findByText('Detalhes do filme')).toBeTruthy();
});
```

## Integração: componente + API mockada

Mocka a camada de dados — testa o componente com dados reais de formato:

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { render, screen } from '@testing-library/react-native';
import MovieList from '../src/screens/MovieList';
import * as movieService from '../src/services/movieService';

jest.mock('../src/services/movieService');
const mockedFetch = movieService.fetchPopularMovies as jest.Mock;

function wrapper({ children }: { children: React.ReactNode }) {
  return (
    <QueryClientProvider client={new QueryClient()}>
      {children}
    </QueryClientProvider>
  );
}

it('exibe lista de filmes retornada pela API', async () => {
  mockedFetch.mockResolvedValue([{ id: 1, title: 'Matrix', vote_average: 8.7 }]);
  render(<MovieList />, { wrapper });
  expect(await screen.findByText('Matrix')).toBeTruthy();
});
```

## renderHook — testando hooks isolados

Quando o hook tem dependências (store, query, context) mas não quer renderizar a tela inteira:

```
import { renderHook, act } from '@testing-library/react-native';
import { useFavorites } from '../src/hooks/useFavorites';

it('toggle adiciona e remove favorito', () => {
  const { result } = renderHook(() => useFavorites());

  act(() => { result.current.toggle(42); });
  expect(result.current.isFavorite(42)).toBe(true);

  act(() => { result.current.toggle(42); });
  expect(result.current.isFavorite(42)).toBe(false);
});
```

`renderHook` é o meio-termo: roda o hook no ambiente React sem precisar de tela.

## Integração: estado persistido entre telas

Testa fluxo completo dentro do React — favoritar em uma tela, verificar em outra:

```
it('favoritar em MovieList reflete no contador do header', async () => {  
  render(<AppNavigator />);  
  
  // Aguarda a lista carregar  
  const card = await screen.findByTestId('movie-card-1');  
  
  // Favorita  
  fireEvent.press(screen.getByTestId('movie-card-heart-1'));  
  
  // Verifica contador  
  expect(screen.getByTestId('favorites-count')).toHaveTextContent('1');  
});
```

Sem simulador, sem Maestro — só React renderizado em memória pelo RNTL.

## Roteiro hands-on Bloco 2 (75min)

**App:** mesmo app do Bloco 1 — agora testando fluxos entre componentes.

1. `renderHook` no `useFavorites` — testar toggle isolado — **15min**
2. `MovieList` com `QueryClientProvider` + API mockada — lista aparece — **20min** ★
3. `MovieList` com `NavigationContainer` — tap navega pra Detail — **20min**
4. Integração completa: favoritar em List → contador atualiza — **15min**
5. Discutir: qual nível de teste cobriu o quê? — **5min**



## Pitfalls testes de integração

- ✗ **Não limpar estado entre testes** — Zustand persiste entre `it()` se não usar `beforeEach`
- ✗ **QueryClient compartilhado** — criar um novo `QueryClient` por teste (cache polui)
- ✗ **`findBy` vs `getBy`** — use `findBy` (async) quando o dado vem de fetch; `getBy` falha imediatamente
- ✗ **Mockar demais** — se você mockou store + nav + api + context, não está testando integração
- ✗ **Ignorar `act()`** — mutations de estado fora de `act()` geram warnings e resultados imprevisíveis

## Atividade 2 — revisão de prazo

A atividade 2 (suíte unitária) foi detalhada agora nesta aula. **Novo prazo: até 21/06.**

| #                   | Entrega                                                                     | Pontos |
|---------------------|-----------------------------------------------------------------------------|--------|
| 1                   | <code>favoritesStore</code> — 6 testes (add/remove/toggle/isFavorite/clear) | 3      |
| 2                   | <code>MovieCard</code> (RNTL) — render + toque navega ★                     | 3      |
| 3                   | <code>isTokenError</code> (data) — 5 testes                                 | 2      |
| 4                   | <code>counterStore</code> — 3 testes                                        | 1      |
| 5                   | Cobertura $\geq 70\%$ em <code>src/store</code> e <code>src/utills</code>   | 1      |
| <b>Eliminatório</b> | <code>npm install &amp;&amp; npm test</code> roda em $< 15\text{min}$       | —      |
| <b>Bônus (+1)</b>   | <code>fetchPopularMovies</code> com <code>jest.mock</code> da api           | +1     |

## Esquenta — Aula 4: Maestro E2E

Na próxima aula vamos sair do RNTL (componentes em memória) e testar o **app real rodando no device**, com Maestro (YAML declarativo).

| RNTL hoje         | → | Maestro na Aula 4             |
|-------------------|---|-------------------------------|
| render em memória |   | app real + simulador/emulador |
| mock de navegação |   | navegação real                |
| mock de API       |   | rede real (ou interceptada)   |
| milissegundos     |   | segundos por teste            |

### Pré-requisito para a Aula 4:

- Simulador iOS (macOS) **ou** emulador Android configurado
- Maestro instalado: `curl -Ls "https://get.maestro.mobile.dev" | bash`

Maestro é simples (YAML, sem build nativo) — instale antes pra ganhar tempo.

## Leituras obrigatórias (pré-Aula 4)

- **Maestro Docs** — *Getting Started* ([maestro.mobile.dev](https://maestro.mobile.dev))
- **mobile.dev Blog** — *Why YAML for E2E Tests*
- **iDFlakies (Lam et al. 2019)** — catálogo de causas de flakiness em E2E

## Próxima aula (22/06)

### Aula 4 — E2E com Maestro

Do RNTL ao app real — YAML declarativo, conditional flows, fragmentos, Maestro Studio, Firebase Test Lab.

Pré-requisito:

- Atividade 2 entregue (21/06)
- Simulador iOS **ou** emulador Android funcionando
- `maestro --version` verde no terminal

# Obrigado!

 [jackson.96@gmail.com](mailto:jackson.96@gmail.com)

 [linkedin.com/in/3jacksonsmith](https://www.linkedin.com/in/3jacksonsmith)

 [github.com/jacksonsmith](https://github.com/jacksonsmith)

**Próxima segunda, 22/06, 19:00**